Pontificia Universidad Católica de Chile
Escuela de Ingeniería
IIC3253 – Criptografía y Seguridad Computacional
Primer Semestre del 2026

Ayudantía 04 – Soluciones

Funciones de Hash

Ayudante: Manuel Cifuentes – `mecifuentes@uc.cl`

Problema 1: Funciones despreciables

(a) $g(n) = f(n) \cdot p(n)$ **es despreciable.**

Queremos mostrar que multiplicar una función despreciable por un polinomio sigue siendo despreciable. Intuitivamente, lo despreciable decrece tan rápido que un polinomio no lo puede salvar.

Sea $q(n)$ un polinomio cualquiera. Queremos mostrar que existe n_0 tal que para todo $n \geq n_0$:

$$g(n) = f(n) \cdot p(n) < \frac{1}{q(n)}.$$

Definimos el polinomio $r(n) = p(n) \cdot q(n)$. Como f es despreciable, existe n_0 tal que para todo $n \geq n_0$:

$$f(n) < \frac{1}{r(n)} = \frac{1}{p(n) \cdot q(n)}.$$

Multiplicando ambos lados por $p(n)$:

$$f(n) \cdot p(n) < \frac{1}{q(n)}.$$

Como q era arbitrario, g es despreciable. $\square$

(b) $f + h$ **es despreciable.**

Sea $p(n)$ un polinomio cualquiera. Queremos que $(f + h)(n) < 1/p(n)$.

Definimos $q(n) = 2 \cdot p(n)$, que es un polinomio.

Como f es despreciable, existe n_1 tal que para todo $n \geq n_1$:

$$f(n) < \frac{1}{q(n)} = \frac{1}{2\,p(n)}.$$

Como h es despreciable, existe n_2 tal que para todo $n \geq n_2$:

$$h(n) < \frac{1}{q(n)} = \frac{1}{2\,p(n)}.$$

Tomando $n_0 = \text{máx}(n_1, n_2)$, para todo $n \geq n_0$:

$$f(n) + h(n) < \frac{1}{2\,p(n)} + \frac{1}{2\,p(n)} = \frac{1}{p(n)}.$$

Como p era arbitrario, $f + h$ es despreciable. $\square$

Problema 2: Resistencia a colisiones implica resistencia a preimagen

(a) Definición del juego $\mathsf{Hash\text{-}PreImg}(n)$.

El juego $\mathsf{Hash\text{-}PreImg}(n)$ se define así:

1. El verificador genera $s = \mathsf{Gen}(1^n)$.
2. El verificador elige m uniformemente al azar de $\{0,1\}^{\ell'(n)}$ y calcula $y = h_s(m)$.
3. El verificador entrega (s, y) al adversario.
4. El adversario produce un mensaje m'.
5. El adversario gana si $h_s(m') = y$.

(Gen, h) es **resistente a preimagen** si para todo adversario PPT $\mathcal{A}$:

$$\Pr[\mathcal{A} \text{ gana } \mathsf{Hash\text{-}PreImg}(n)] \leq \mathsf{negl}(n).$$

Alternativa válida: también se puede definir eligiendo y uniformemente de $\{0,1\}^{\ell(n)}$. Ambas formulaciones son aceptables; la primera es más estándar (Katz–Lindell).

(b) Resistencia a colisiones $\Rightarrow$ Resistencia a preimagen.

Procedemos por **CONTRADICCIÓN**. Supongamos que (Gen, h) **no** es resistente a preimagen. Entonces existe un adversario PPT $\mathcal{A}$ que corre en tiempo $q(n)$ para algún polinomio q, y existe un polinomio $p(n)$ tal que:

$$\Pr[\mathcal{A} \text{ gana } \mathsf{Hash\text{-}PreImg}(n)] \geq \frac{1}{p(n)}$$

para infinitos valores de n.

Construcción del adversario $\mathcal{B}$ que rompe colisiones:

$\mathcal{B}$ recibe s del verificador del juego $\mathsf{Hash\text{-}Col}(n)$ y hace lo siguiente:

1. Elige $m^* \in \{0,1\}^{q(n)+1}$ uniformemente al azar.
2. Calcula $y = h_s(m^*)$.
3. Ejecuta $\mathcal{A}$ con entrada (s, y). $\mathcal{A}$ devuelve m'.
4. Si $h_s(m') = y$, entonces $\mathcal{B}$ devuelve (m^*, m') como colisión.

Análisis:

- $\mathcal{A}$ corre en tiempo $q(n)$, por lo que su salida m' tiene largo a lo más $q(n)$. En cambio, m^* tiene largo $q(n)+1$. Por lo tanto, $m' \neq m^*$ con probabilidad 1: simplemente no tienen el mismo largo.
- $\mathcal{A}$ devuelve m' con $h_s(m') = y = h_s(m^*)$ con probabilidad $\geq 1/p(n)$.
- Combinando ambos puntos: cuando $\mathcal{A}$ tiene éxito, automáticamente tenemos $h_s(m') = h_s(m^*)$ con $m' \neq m^*$, que es una colisión.
- Por lo tanto:
$$\Pr[\mathcal{B} \text{ gana } \mathsf{Hash\text{-}Col}(n)] \geq \frac{1}{p(n)},$$
que no es despreciable. Esto contradice que (Gen, h) es resistente a colisiones. $\square$

Problema 3: Construcción XOR de funciones hash

Sea (Gen', h') una función hash resistente a colisiones con salida de largo $\ell(n)-1$. Definimos dos funciones hash con el mismo largo de salida $\ell(n)$: para una entrada de la forma $b\|x$ donde $b \in \{0,1\}$ y $x \in \{0,1\}^*$, definimos:

$$h_1^s(b\|x) = b\|h'^s(x), \qquad h_2^s(b\|x) = b\|h'^s(x).$$

Tomamos $Gen_1 = Gen_2 = Gen'$.

Ambas son resistentes a colisiones: si encontramos una colisión $b_1\|x_1 \neq b_2\|x_2$ con $b_1\|h'^s(x_1) = b_2\|h'^s(x_2)$, entonces $b_1 = b_2$ y $h'^s(x_1) = h'^s(x_2)$ con $x_1 \neq x_2$, lo que es una colisión en h'.

La construcción XOR no es resistente a colisiones: notar que Gen ejecuta Gen' dos veces de forma independiente, obteniendo claves s_1 y s_2 (posiblemente distintas). Aun así:

$$h_{s_1,s_2}(b\|x) = \big(b\|h'^{s_1}(x)\big) \oplus \big(b\|h'^{s_2}(x)\big) = (b \oplus b)\|\big(h'^{s_1}(x) \oplus h'^{s_2}(x)\big) = 0\|\big(h'^{s_1}(x) \oplus h'^{s_2}(x)\big).$$

El primer bit de la salida es siempre 0, independiente de b. Por lo tanto, los mensajes $m_1 = 0\|x$ y $m_2 = 1\|x$ (con $m_1 \neq m_2$) siempre producen la misma salida:

$$h_{s_1,s_2}(0\|x) = h_{s_1,s_2}(1\|x),$$

lo que constituye una colisión trivial para cualquier x y cualesquiera claves s_1, s_2. $\square$